

Pasos de montaje

Runbook del sprint — 10 días hábiles
Digi Starlink-Primary Resilient Gateway Fleet

Capstone de internship · Digi International | Julio 2026

Runbook **día a día** del sprint de 2 semanas (Anillo 0), con el enfoque **Digi primero**. El track embebido corre **en paralelo** desde el Día 1 porque los builds de Yocto son lentos y no deben bloquear el resto. Al final está la fase 2 (Opengear + expansión).

Cada día declara: **objetivo, pasos, verificación (done)** y **track embebido en paralelo**. Nada se marca “done” sin la verificación observada. No se adivinan contratos de API: donde aparezca un gate G*, se cierra antes de depender de él (ver `07-riesgos-y-verificaciones.md`).

Ritmo: 10 días hábiles. Cada día trae un *stretch* opcional. Si el hardware de sitios 2-3 u Opengear llega durante el sprint, se salta a la sección de fase 2 sin reordenar lo demás.

Índice

1. Día 0 — Charter y gates (antes de tocar hardware)	1
2. Semana 1 — Fundamentos Digi + arranque embebido	2
2.1. Día 1 — Sitio A físico + Starlink online + arranque del build Yocto	2
2.2. Día 2 — 5G como segundo underlay + primer boot embebido	2
2.3. Día 3 — Fibra de servicio + overlay + eBGP	3
2.4. Día 4 — DRM-as-code + alarmas + Push Monitor → SIEM	3
2.5. Día 5 — SureLink multicapa + primera campaña de failover nativo	4
3. Semana 2 — Resiliencia avanzada, embebido a producción, clonado, pruebas, demo	4
3.1. Día 6 — WAN Bonding A/B + endurecimiento del agente embebido	4
3.2. Día 7 — Telemetría XBee + Digi Container + arranque de OTA	5
3.3. Día 8 — Clonado de sitio con automatización	5
3.4. Día 9 — Campaña de caos + seguridad + congelar datos	5
3.5. Día 10 — Empaquetado y demo	6
4. Fase 2 — Opengear + expansión (cuando llegue el hardware)	6
4.1. F2-A · OOB / Lighthouse	6
4.2. F2-B · Tres sitios + rotación + 30 reps	6
4.3. F2-C · Embebido a producción	7
5. Notas de secuenciación	7

1 Día 0 — Charter y gates (antes de tocar hardware)

- Objetivo:** dejar el proyecto listo para comprar/montar sin sorpresas.
- Aprobar objetivos, límites de divulgación y reglas de publicación con el manager (ver `08-executive-pitch-EN.md`).
 - Congelar **firmware** de IX40, Hive y MP255 → matriz de versiones.
 - License audit (G8):** confirmar en el tenant DRM Premier + Containers + Push Monitor + CCCS/firmware repo.

4. Cotizar/pedir hardware Digi del sprint y **autorizar en paralelo** sitios 2-3 y Opengear (fase 2).
5. Contratar 1 plan Starlink Priority y 1 SIM 5G Carrier A; validar cobertura de banco (**G10**).
6. Cerrar o poner en curso **G3** (body de `scan_now`), **G4** (nombre del campo ASN remoto), **G9** (plan/HW Starlink), **G12** (`type/vendor_id` del IX40). Los gates de Opengear (**G1/G2/G11**) van en paralelo, no bloquean.

Done: matriz de versiones congelada; SKUs y licencias cotizables/reconciliados; plan Starlink y SIM contratados; host de build DEY provisionado.

2 Semana 1 — Fundamentos Digi + arranque embebido

2.1 Día 1 — Sitio A físico + Starlink online + arranque del build Yocto

Objetivo: conectividad básica del sitio y el primer **bitbake** corriendo.

Pasos (in-band):

1. Rack del Sitio A, energía en PDU/UPS primaria, etiquetado.
2. Starlink Performance Kit → IX40 WAN/ETH1 (cobre). Dejar SFP/ETH1 vacío.
3. IX40 arranca; acceso local por HTTPS/SSH (no exponer a Internet).
4. Registrar el underlay Starlink observado (**G9**): CGNAT 100.64.0.0/10 vs IPv4 pública, IPv6 /64 SLAAC y /56 DHCPv6-PD, MTU, latencia/jitter base.

Done: el IX40 alcanza Internet por Starlink (IPv4 y, si se entrega, IPv6); ficha del underlay registrada.

Track embebido (paralelo):

- Provisionar host de build; instalar dependencias de **Digi Embedded Yocto (DEY 4.0)**.
- `repo/git` clone de las capas `meta-digi`; `mkproject.sh` para plataforma **ccmp25** (ConnectCore MP255).
- Lanzar el **primer bitbake** `<dey-image-*>` (tarda **horas**; dejar corriendo toda la noche).

2.2 Día 2 — 5G como segundo underlay + primer boot embebido

Objetivo: WAN de respaldo lista y la imagen custom arrancando en el MP255.

Pasos (in-band):

1. Configurar el módem 5G del IX40 con SIM/APN de Carrier A → segundo underlay in-band.
2. Métricas WAN: ETH1 métrica IPv4 = 1 (Starlink, prioritaria), `Modem` = 3.
3. SureLink **básico** en ambas interfaces (sin afinar todavía).
4. Registrar IPv4/IPv6/MTU/NAT del 5G (**G10**).

Done: al desconectar Starlink a mano, el tráfico sale por 5G (verificación gruesa, sin medir aún).

Track embebido (paralelo):

- Flashear la imagen del Día 1 en el MP255; **primer boot**.
- Validar arranque, consola, red del SOM. Guardar la imagen base reproducible.

2.3 Día 3 — Fibra de servicio + overlay + eBGP

Objetivo: el sitio anuncia su red por BGP sobre el overlay al core.

Pasos (in-band):

1. Poblar SFP/ETH5 con la óptica **1 Gb/s validada (G6)** → fabric/carga de servicio. **Nunca SFP+**.
2. Definir la red de servicio del sitio: IPv4 RFC1918 /24 + IPv6 documentación /64 (2001:db8::/32).
3. Levantar el **overlay IKEv2/IPsec route-based con NAT-T** hacia el endpoint central público (NAT-T obligatorio por CGNAT).
4. Configurar **eBGP** sitio (AS65001) ↔ core (AS65000) bajo **network route service bgp**. Capturar el nombre real del campo de **ASN remoto** del vecino inspeccionando el esquema DAL (? en **network route service bgp neighbour** o la API local <https://{ix40}/cgi-bin/config.cgi> — **G4**, no adivinar.
5. Aplicar políticas: prefix-lists in/out, max-prefix, prohibir anunciar default.

Done: el collector AS65000 recibe **exactamente** los prefijos IPv4/IPv6 esperados del sitio; los filtros negativos pasan (no se filtra lo prohibido); MTU documentada; cero route leaks.

Track embebido (paralelo):

- Crear la **receta custom** en una capa propia (meta-<lab>) que empaquete el **agente** (Python) en la imagen.
- Rebuild incremental (mucho más rápido que el primero).

2.4 Día 4 — DRM-as-code + alarmas + Push Monitor → SIEM

Objetivo: el sitio se gestiona como código y emite eventos al SIEM.

Pasos:

1. **Enrolar** el IX40 por API: `POST /ws/v1/devices/inventory?bundle_device=true` con `id` + `install_code` (de la etiqueta, nunca en el repo).
2. Importar la config del IX40 de referencia, leer el árbol real y crear la **golden config** con Configuration Manager: `POST /ws/v1/configs/inventory` (campos `name`, `groups`, `type`, `vendor_id`, `enabled`, `alert`, `remediate`, `scan_frequency`, `maintenance_window_handling`...). **Obtener type/vendor_id del IX40 real (G12); no copiar el ejemplo TX64.**
3. Settings comunes vs por sitio (`/settings` y `/settings/device?device_id=...`); recordar que un PUT sustituye el árbol enviado → declarar solo nodos administrados (`@managed`, `@scope`, `#value`).
4. Validar el body de `scan_now` contra `GET /ws/v1/configs` o el API Explorer del tenant (**G3**); iniciar un scan y consultar `GET /ws/v1/configs/compliance`.
5. Alarmas: `POST /ws/v1/alerts/inventory` (Device Offline con `fire.parameters.reconnectWindowDuration`) + `noncompliance`.
6. **Push Monitor:** `POST /ws/v1/monitors/inventory` (`type: http`, `topics: [alert_status]`, `url` al SIEM, `persistent: true`).

Done: flujo completo observado con contratos validados: enrolamiento, golden config aplicada, scan/compliance, alarma disparada y evento recibido por el SIEM vía Push Monitor.

Track embebido (paralelo):

- El agente recolecta **red local + ambientales** y envía al SIEM (formato estructurado, mismo reloj NTP, ID por evento).

2.5 Día 5 — SureLink multicapa + primera campaña de failover nativo

Objetivo: failover afinado y **medido** (no “ping ok”).

Pasos:

1. SureLink con pruebas **multicapa** e independientes: **interface_up**, ping de gateway, ping externo, **dns**, **https** y equivalentes IPv6. Múltiples destinos (evitar punto único de fallo).
2. Acciones **escalonadas**: subir métrica → reiniciar interfaz; reset de módem/cambio de SIM solo tras fallos persistentes.
3. **Histéresis + hold-down** contra flapping; **failback** a Starlink solo tras ventana estable.
4. Montar el testigo de medición: ping continuo + una videollamada/stream como carga real.
5. Ejecutar la **campana representativa** (≈ 10 reps): desconexión física de Starlink y blackhole aguas arriba (Ethernet up). Correlacionar cada inyección con un ID único.

Done: para desconexión y blackhole, se tiene tiempo de detección, tiempo de conmutación y pérdida por repetición, con p50/p95; trazas correlacionadas con la alerta DRM.

Track embebido (paralelo):

- Implementar **buffer offline + reenvío** del agente: al cortar la red, sigue recolectando; al reconectar, reenvía sin perder datos.

Stretch: Semana 1: detección ligera de anomalías en el agente (umbral/estadística simple sobre las series recolectadas).

3 Semana 2 — Resiliencia avanzada, embebido a producción, clonado, pruebas, demo

3.1 Día 6 — WAN Bonding A/B + endurecimiento del agente embebido

Objetivo: comparar los dos mecanismos de resiliencia y endurecer el agente embebido (secure boot se difiere a fase posterior).

Pasos (in-band):

1. Aprovisionar el **servidor externo de WAN Bonding** (Tier 3, 1 Gb/s) con credenciales de túnel.
2. En el IX40: `network sdwan wan_bonding`, meter ETH1 y Modem en el túnel, cifrado on. Estado con `show wan-bonding`.
3. Reproducir el **mismo impairment trace** del Día 5 y comparar native vs bonding: continuidad de sesión, throughput, RTT, jitter, pérdida.

Done: informe comparativo native vs bonding con el mismo trace; la sesión de la app de prueba **no** se resetea bajo bonding (objetivo pérdida $\leq 1\%$ en handoff).

Track embebido (paralelo):

- **Endurecer el agente + validar resiliencia:** afinar la detección de anomalías y repetir cortes de red bajo carga para confirmar buffer/replay sin pérdida.

- *TrustFence secure boot se movió a fase posterior (F2-C) por ser irreversible (trustfence close) y de tiempo impredecible.*

3.2 Día 7 — Telemetría XBee + Digi Container + arranque de OTA

Objetivo: cerrar la telemetría física y el edge compute del router.

Pasos:

1. Emparejar el sensor **XBee 3 Zigbee** con el **Hive**; publicar la lectura (temperatura/puerta/entrada digital) como **DataPoints** en DRM.
2. Desplegar un **Digi Container** ligero (collector de métricas) al IX40 desde DRM, con **límites de recursos**.
3. Prueba de carga del container: demostrar que **no degrada** el plano de forwarding.

Done: la telemetría llega por ambos underlays; uso de recursos del container medido y sin impacto en el forwarding.

Track embebido (paralelo):

- Configurar **OTA** vía ConnectCore Cloud Services: `/etc/cccs.conf`, construir un paquete `.swu`, subirlo al **firmware repository de Remote Manager** (p.ej. con `python-devicecloud`) y disparar una actualización de prueba. (*Stretch del sprint; completo en Anillo 2.*)

3.3 Día 8 — Clonado de sitio con automatización

Objetivo: demostrar que un sitio nuevo se levanta con un comando.

Pasos:

1. Modelar la config como **datos**: plantillas Jinja2 + variables por sitio (ASN, prefijos, tunnel peer, nombres, carrier).
2. Envolver todo en Ansible/Python + **make deploy**: enrolamiento API → golden config por grupo → settings por sitio → verificación.
3. Levantar el **Sitio B**:
 - Si llegó el hardware → sitio **físico** clonado.
 - Si no → clonar contra un dispositivo de evaluación/simulado y **documentar** que la limitación es de hardware, no de método.
4. Probar **idempotencia**: correr **make deploy** dos veces no rompe nada.

Done: un segundo sitio gestionado surge del comando; el registro de auditoría (vía RADIUS/TACACS+ → SIEM) muestra cada acceso trazado; idempotencia demostrada.

Stretch: GitOps — un cambio en Git dispara la reconciliación de la config de la “flota”.

3.4 Día 9 — Campaña de caos + seguridad + congelar datos

Objetivo: ejercer un subconjunto de la matriz de fallos y dejar la evidencia limpia.

Pasos:

1. Ejecutar los fallos del sprint (ver `06-pruebas-y-demo.md` §Matriz): desconexión Starlink, blackhole, DNS-only, drift de config, receiver de webhook offline.
2. Correlacionar **alarma** ↔ **evento Push Monitor** ↔ **ID de fallo**; probar **replay** desde `GET /ws/v1/monitors/history/{monitorId}` tras dejar el receiver caído.

3. Drift + **remediación** por Configuration Manager dentro de una maintenance window.
4. Revisión de seguridad: usuarios de aplicación separados, API keys con vencimiento, MFA/R-BAC, cero defaults, **cero secretos** en artefactos; sanitizar evidencia.

Done: todos los casos del sprint tienen resultado, evidencia y dueño; replay comprobado; drift remediado; datasets congelados con versión de firmware/config.

3.5 Día 10 — Empaquetado y demo

Objetivo: dejar el capstone entregable.

Pasos:

1. Diagrama **lógico** y **as-built**; BOM validada; documento de intención de configuración por firmware.
2. Contrato de API DRM validado contra el tenant real (y Lighthouse en fase 2).
3. Runbook repetible (onboarding, failover, OOB, rollback, recuperación).
4. Grabar la **demo de 12 min** + versión de **90 s**; ensayar (ver guion en 06-pruebas-y-demo.md §Demo).
5. **One-pager ejecutivo** para el manager + **README sanitizado** para portafolio.

Done: demo de 12 min ensayada; paquete entregado al manager; versión pública sanitizada lista; backlog de gaps (G1–G12) con dueño recomendado.

4 Fase 2 — Opengear + expansión (cuando llegue el hardware)

Estos bloques **no** están en el camino crítico del sprint; se ejecutan cuando el hardware Opengear y las unidades 2-3 llegan. Mapea a las semanas 5–10 de plan2.

4.1 F2-A · OOB / Lighthouse

1. Cerrar **G1** (confirmar SKU 0M1304-4E-C-LSP, revisión, región, firmware) — o activar fallback CM8004-C-LSP + switch externo.
2. Enrolar el OM por **LSP** en Lighthouse Enhance (POST /nodes con enrollment; Authorization: Token {session}). Probar **factory reset/boot** para demostrar zero-touch (systemctl {start,enable} lsp).
3. Consola serial del IX40 en modo **Login** (adaptador 319015), acceso restringido por RBAC.
4. Validar runtime_status.connection_status y **acceso a consola durante pérdida total del plano in-band**.
5. Secure Provisioning downstream: **opcional**, hasta confirmar licencia en Core/Enhance (**G2/G11**); observar progreso en syslog [NetOps-D0P device="MAC"].

4.2 F2-B · Tres sitios + rotación + 30 reps

1. Clonar Sitios B y C físicos con la automatización del Día 8.
2. Rotar perfiles **native/bonding/candidate** entre las tres unidades.
3. Campaña de **30 repeticiones** por fallo con p50/p95/p99 y datos brutos.

4.3 F2-C · Embebido a producción

1. **TrustFence secure boot:** generar claves, SRK_efuses.bin, firmar U-Boot + kernel, trustfence close (**irreversible**; practicar en unidad sacrificable), cifrado de rootfs (CONFIG_FS_ENCRYPTION, datos en /mnt/data/private).
2. **OTA completo** del agente (.swu por CCCS) sin ladrillar el dispositivo.
3. Carril **OEM** con el MP255 (TSN/NPU) en Site C.
4. Lazo de **auto-remediación**: árbol de decisión según tipo de fallo, verificación post-remediación y rollback.

trustfence close es irreversible. Ejecutar únicamente con la imagen ya estable y en una unidad sacrificable de prueba antes de aplicar a producción.

5 Notas de secuenciación

- **Yocto arranca el Día 1** porque el primer build tarda horas; si esperara a la semana 2, bloquearía el track central.
- **DRM-as-code (Día 4) antes del clonado (Día 8):** no puedes clonar lo que aún no modelaste como datos.
- **SureLink medido (Día 5) antes de Bonding (Día 6):** necesitas el baseline nativo para que la comparación A/B signifique algo.
- **trustfence close es irreversible:** por eso **no** entra en el sprint; se ejecuta en fase posterior (F2-C) con la imagen ya estable.
- **Opengear fuera del camino crítico:** su retraso no puede frenar el sprint; entra completo en fase 2.